

第二十一届全国大学生
智能汽车竞赛

技 术 报 告

学 校：_____

队伍名称：_____

参赛队员：_____

带队教师：_____

关于技术报告和学术论文使用授权的说明

本人完全了解全国大学生智能汽车竞赛关于保留、使用技术报告和学术论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编辑收录在组委会出版论文集中。

参赛队员签名：_____

带队教师签名：_____

日 期：_____

目录

第一章 引言	1
第二章 系统总体方案	2
2.1 系统组成.....	2
2.2 主流程与状态机.....	3
2.3 运行模式.....	3
第三章 机械结构	4
3.1 底盘机械结构.....	4
3.2 传感器方案.....	4
3.3 安装与标定.....	4
第四章 电路设计	6
4.1 主控与供电.....	6
4.2 电机与编码器接口.....	6
第五章 算法设计	7
5.1 软件调度与算法数据流.....	7
5.2 网格寻路与炸弹清障.....	7
5.3 巡图观测与在线宏动作规划.....	8
5.4 语义绑定与推箱解算.....	8
5.5 路径跟踪与平移速度规划.....	9
5.6 麦克纳姆运动学与轮速闭环.....	10
5.7 航向、位姿融合与安全控制.....	11
第六章 开发、制作与调试	13

6.1 开发工具.....	13
6.2 安装与调试过程.....	13
6.3 故障保护.....	13
6.4 测试记录.....	13
第七章 主要技术参数	15
第八章 结论	16
参考文献	i
附录 A 核心程序	ii
附录 B 电路板原理图	iii
附录 C 提交前检查表	iv

第一章 引言

本项目面向第二十一届全国大学生智能汽车竞赛智能视觉组推箱子赛题，完成了一辆能够自主建图、巡图识别、清障、推箱和返航的模型汽车。模型车以四轮麦克纳姆轮底盘为机械主体，制作过程中完成了车架与轮组装配、四路电机和增量编码器安装、RT1064 主控板及电机驱动与供电线路连接，并安装 ICM42688 惯性测量单元、两路 OpenART 视觉模块和 TFT 显示模块。ART1 用于地图与全局位姿获取，ART2 用于箱子和目标点语义识别，各模块共同构成车辆的环境感知与运动执行系统。

整车控制软件围绕比赛任务建立分层结构。主控接收 ART1 地图后生成逻辑关卡，Strategy 与 Exploration 规划炸弹清障和巡图观测顺序，MacroPlanner 根据实时地图和已识别语义在线派发观测、推炸弹及推箱宏动作，Sokoban 求解器生成最终推箱路径，Tracker、位姿估计和四轮速度闭环负责将网格路径转换为实际运动。比赛状态机将出库、建图、巡图、语义绑定、推箱、返航和异常停车串联起来，并提供实车、Mock 和 Demo 三种运行模式用于现场执行与调试。

相关智能汽车方案通常采用编码器里程计和 IMU 进行短时运动估计，再利用视觉定位抑制长期累计误差；复杂任务规划则常使用网格搜索、启发式搜索和状态剪枝降低求解规模。本项目在这些方法的基础上，根据推箱子赛题增加了炸弹清障、N-1 语义绑定、动态宏动作调度以及 IDA* 推箱搜索，并针对 RT1064 的片上存储限制使用固定容量数组和分区缓存，避免动态内存进入规划与控制热路径。

设计制作和调试采用 Keil MDK/armclang 完成嵌入式构建，桌面仿真程序用于地图回归和规划算法检查。后续各章依次说明系统总体方案、机械与传感器安装、电路及通信、控制软件与算法、开发调试过程和主要技术参数，并在结论中总结实车调试问题、当前解决方法和进一步改进方向。

第二章 系统总体方案

2.1 系统组成

系统分为 App、Algorithm、Subsystem、Device 和 Core 五层。App 层负责比赛状态机；Algorithm 层负责规划、求解和控制算法；Subsystem 层连接业务与设备；Device 层封装电机、编码器、IMU 和串口；Core 层提供中断、调度与参数配置。

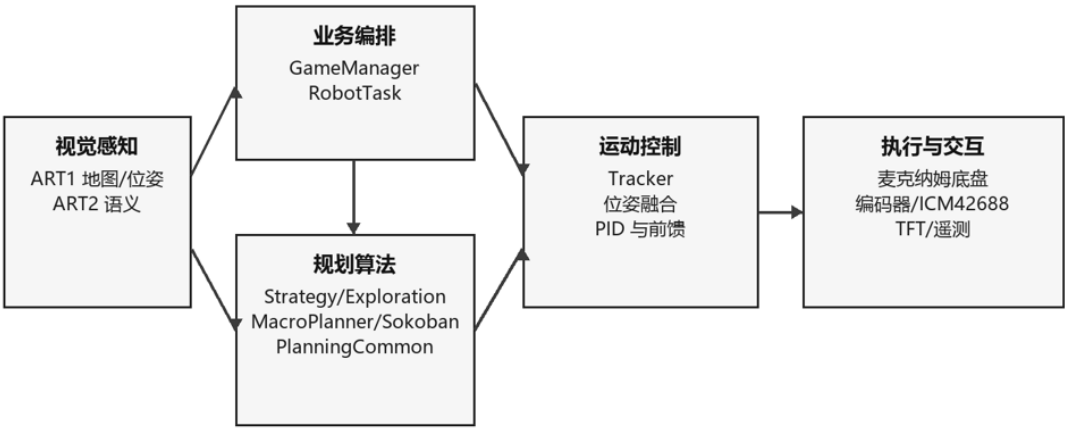


图 2.1 智能视觉组主控系统总体架构

表 2.1 软件分层及职责

层级	核心模块	主要职责
App	GameManage、RobotState、RobotTask	比赛流程和任务队列
Algorithm	PlanningCommon、Strategy、Exploration、MacroPlanner、Sokoban	寻路、巡图、语义调度和推箱
Subsystem	Vision、ChassisControl、PoseEstimate	通信、底盘与位姿融合
Device	UartComm、Motor、Encoder、Icm42688	设备驱动与抽象
Core	CoreScheduler、isr、配置文件	周期调度、中断和系统参数

2.2 主流程与状态机

ART1 地图就绪后，GameManager 将视觉数据写入 logical_level。高级赛段先由 Strategy 计算炸弹任务，Exploration 生成巡图参考序列，再由 MacroPlanner 在线选择观测、推炸弹或完成式推箱动作。ART2 语义结果满足绑定条件后，Sokoban 求解最终路径，Tracker 和底盘完成执行与返航。

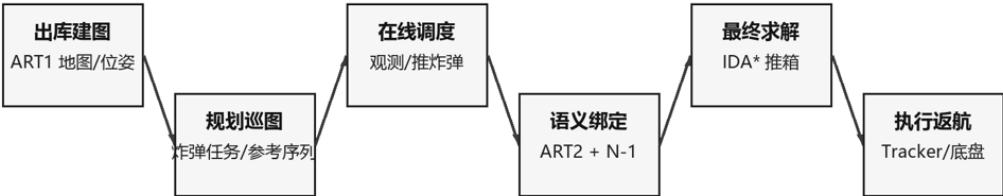


图 2.2 比赛主流程与异常回退

2.3 运行模式

表 2.2 系统运行模式

模式	视觉输入	底盘动作	用途
Prod	真实 ART1/ART2	真实执行	正式比赛
Mock	注入地图和语义	真实执行	脱机联调
Demo	注入语义	动画推演	状态机演示

第三章 机械结构

3.1 底盘机械结构

车模采用四轮麦克纳姆轮全向底盘，轮半径为 3.15 cm，x 方向半轴距为 9.0 cm，y 方向半轴距为 10.0 cm。正式提交前需加入实车正视图、俯视图和传感器安装图。

表 3.1 底盘机械参数

项目	设计值	提交前核验
轮型	四轮麦克纳姆	轮辋方向及安装照片
轮半径	3.15 cm	实测有效半径
半轴距	X=9.0 cm, Y=10.0 cm	装配后中心距
整车尺寸	待填写	长×宽×高
整车质量	待填写	含电池和视觉模块

3.2 传感器方案

四路增量编码器用于轮速反馈与里程计，ICM42688 用于姿态和角速度测量。ART1 通过 UART 返回地图和位姿，ART2 负责箱子与目标点的语义识别。PoseEstimator 按稳定帧、方差与时间新鲜度条件融合视觉位姿。

表 3.2 传感器配置

传感器	数量	接口或周期	用途
增量编码器	4	正交计数；5/20 ms	轮速和里程计
ICM42688	1	1 ms 采样	姿态与角速度
OpenART (ART1)	1	UART4, 115200 bps	地图和位姿
OpenART (ART2)	1	UART1, 115200 bps	语义识别
TFT	1	平台显示接口	状态与调参

3.3 安装与标定

- 核对四轮电机和编码器的软件符号与实际方向
- IMU 上电静态标定期间保持车辆静止，并核验 yaw 零点
- 记录 ART1、ART2 相机高度、俯仰角、视场方向和相机坐标系
- 到达观测点后先原地对准并确认停稳，再触发 ART2 抓拍

第四章 电路设计

4.1 主控与供电

主控采用 NXP i.MX RT1064 核心板。工程按 ITCM、DTCM 和 OCRAM 分区：小型实时对象放入 DTCM，大型规划缓存放入 OCRAM，只有明确需要的函数放入 ITCM。供电电压、降压模块、保护器件、系统功耗和总电容须依据最终电路板和实测数据补齐。

4.2 电机与编码器接口

Motor 类封装方向 GPIO、PWM 占空比限幅和安装方向反转。编码器物理线数为 1024，硬件四倍频，传动比为 70/30，轮端每转脉冲数由 $1024 \times 4 \times 70/30$ 计算。

表 4.1 已确认的接口配置

模块	接口或引脚	说明
LF 电机	C6 + PWM2_MODULE0_CHB_C7	方向 GPIO + PWM
LB 电机	C11 + PWM2_MODULE2_CHA_C10	方向 GPIO + PWM
RF 电机	C8 + PWM2_MODULE1_CHB_C9	软件方向反转
RB 电机	D3 + PWM2_MODULE3_CHA_D2	软件方向反转
ART2	UART1, B12/B13, 115200	语义链路
ART1	UART4, C16/C17, 115200	地图和位姿链路

第五章 算法设计

5.1 软件调度与算法数据流

控制软件采用“比赛状态机—宏动作规划—底层任务—实时控制”四级结构。GameManager 根据地图、语义和当前比赛阶段决定业务流程，Strategy、Exploration、MacroPlanner 与 Sokoban 只处理规划问题；宏动作随后被拆分为加载路径、等待跟踪、原地对准和触发视觉等 RobotTask，最终由 Tracker、PoseEstimator 和 ChassisControl 执行。该结构把计算量较大的离散规划与固定周期的连续控制隔离，避免搜索过程直接占用中断控制时间。

系统使用 1 ms GPT 作为软件调度时基，并由 PIT 中断运行不同周期的实时任务。IMU 姿态更新频率最高，轮速控制以 5 ms 快环运行；路径跟踪、速度规划和里程计使用 20 ms 慢环；视觉坐标矫正独立运行在 10 ms 周期。较慢的上位机通信和 TFT 刷新放入主循环任务表，从而保证电机控制和姿态估计的确定性。

表 5.1 控制软件主要周期任务

任务	周期	输入	主要输出
IMU 采样与姿态解算	1 ms	陀螺仪、加速度计	yaw 与角速度
四轮速度闭环	5 ms	目标与实测轮速	四路 PWM 占空比
视觉位姿修正	10 ms	ART1 位姿、里程计历史	融合位姿
路径与底盘慢环	20 ms	航点、当前位姿	底盘速度与轮速目标
上位机指令解析	10 ms	遥测串口	调参和控制指令
波形数据发送	20 ms	全局运行状态	上位机波形
TFT 刷新	100 ms	调试与比赛状态	界面显示

5.2 网格寻路与炸弹清障

规划层把 240 cm×320 cm 场地离散为 12×16 网格，每格边长 20 cm。SokobanLevel 保存静态墙体、箱子、目标点、炸弹和玩家位置，动态物体与墙体分开维护。PlanningCommon 在普通最短路径的基础上增加朝向状态和转向

时间代价，使路线评价不仅考虑格数，还能减少频繁停车和方向切换。搜索缓存采用固定尺寸数组和 `StaticArray`，避免在 RT1064 上产生动态内存碎片。

箱子和炸弹不能被当作普通障碍简单绕过，因此规划器先计算玩家可达区域，再判断推动方向后的支撑位置、物体目标格和后续可达性。`Strategy` 分两阶段选择炸弹任务：巡图前优先打通观测和移动所需通道，语义绑定后再结合箱子—目标配对重新评估剩余墙体。每次推炸弹后，`GameManager` 更新逻辑地图并等待爆炸窗口结束；在此期间暂时抑制视觉修正，防止爆炸闪光产生错误位姿。

5.3 巡图观测与在线宏动作规划

`Exploration` 根据每个箱子和目标点的相邻可站立格生成 `ViewPose` 候选，候选同时包含观测位置、朝向和可覆盖实体掩码。目标点观测可在一次抓拍中排列最多三个实体，规划器据此减少重复停车。离线参考阶段综合路径时间、转向代价、炸弹执行顺序和顺路观测机会，生成一条可执行的巡图参考序列。

实际执行时环境状态会因推箱、炸弹爆炸和语义识别结果发生变化，因此 `MacroPlanner` 不机械照搬参考序列，而是每次根据 `MacroPlanContext` 在线选择下一动作。上下文由 `GameManager` 填入真实 `logical_level`、当前位置、当前 `yaw`、已观测掩码和炸弹任务，规划器只保存知识状态而不复制真实世界地图。当参考动作不可达时，系统会尝试重新寻路、插入清障动作或调用 `Exploration` 兜底，降低局部状态变化导致整条计划失效的风险。

宏动作最终被拆为底层任务队列：观测动作包括加载观测路径、等待到位、原地调整 `yaw` 和等待 ART2 抓拍；炸弹或箱子动作包括路径展开、跟踪等待和地图更新。只有车辆位置、航向和静止状态同时满足条件时才触发识别，避免运动模糊和相机视角偏差污染语义结果。

5.4 语义绑定与推箱解算

ART2 返回的语义标签先写入统一实体池，其中箱子编号位于前半部分，目标点编号紧随其后，-1 表示尚未识别。一次观测只有在抓拍 ACK 和请求掩码内全部实体结果到齐后才结算，避免旧帧或部分回包提前改变配对状态。`MacroPlanner` 每次派发新动作前同步该语义池，并检查当前信息是否足以进入最终推箱阶段。

语义绑定先匹配已识别且标签相同的箱子和目标点，再应用 N-1 补全规则：若仅剩一个未识别目标点，则将已识别但尚无同标签目标的箱子与其配对；仅剩一个未识别箱子时反向处理；最后只允许唯一剩余箱子与唯一剩余目标互补。该规则在减少观测次数的同时保持配对唯一性，若仍存在多解则继续巡图，不把不确定关系交给求解器。

绑定完成后，GameManager 将 box→target 配对写入 logical_level，重新计算第二阶段炸弹任务并加载 Sokoban 求解器。求解状态包含玩家规范位置、各箱子位置和语义、炸弹状态及未完成目标集合；启发式代价根据箱子到兼容目标的距离建立下界。求解器采用 IDA* 逐步提高代价阈值，并结合死锁检测、Zobrist 哈希和置换表剪去重复状态。

为降低逐步搜索深度，求解器还生成可行推箱宏动作和炸弹宏动作，再由 PlanningCommon 展开为玩家实际行走路径。候选解完成后必须重新模拟箱子、炸弹和玩家状态，验证路径连续性 & 目标语义一致性。若最终求解失败，prepare_phase2_solver 会根据当前地图重新绑定语义和规划炸弹任务；仍无法求解时返回感知阶段，而不是执行未经验证的路径。

5.5 路径跟踪与平移速度规划

Tracker 接收规划层输出的逐格路径后，首先利用真实逻辑起点参与共线判断，只保留拐点和终点，再把网格坐标转换为物理坐标。该处理可减少 20 cm 网格路径中的冗余航点，同时避免原始路径没有显式包含起点时首段方向判断错误。推箱、推炸弹和普通观测共用同一跟踪框架，但可以携带强制停车、持续视觉辅助等不同属性。

跟踪过程中 current_wp_idx 指向当前物理航点，segment_start 保存当前直线段起点。普通航点使用到达窗口和横向误差共同判断是否切段，最终目标则要求进入更严格半径并确认四轮已经停稳。corner_pass_speed 可控制拐点处的段末保留速度；当前默认值为 0 cm/s，因此以稳定到点为优先。任务切换或到达终点时置位 hard_lock，底盘目标立即归零并清除速度规划残留，防止上一段速度带入下一动作。

PathLineFollower 不是简单朝目标点行驶，而是将当前位置投影到 segment_start—target 直线，根据沿线距离选择动态前瞻点，并用法向误差生成

横向拉回速度。前瞻距离随当前速度变化，方向指令再经过一阶滤波，减少切段时速度矢量突变；当横向误差增大时，系统主动降低沿线速度并提高纠偏比例，以避免高速状态下越修越偏。

Trajectory 根据剩余距离、段长、最大速度、加速度和实际制动能力生成平滑速度。默认最大平移速度为 200 cm/s、最大加速度为 800 cm/s²，制动能力另受 450 cm/s² 上限约束，防止仅提高加速度参数后错误地缩短刹车窗口。长段巡航速度与减速距离由可用段长和制动能力反推，短段提高起步加速度但不虚增刹车能力；停车段末端采用随剩余距离下降的速度曲线，使车辆在视觉冻结窗口前完成主要减速。

5.6 麦克纳姆运动学与轮速闭环

路径跟踪器输出全局坐标系下的期望速度，底盘控制先依据当前 yaw 将其旋转到车体坐标系。项目约定车体 x 轴指向右侧、y 轴指向前方，随后通过麦克纳姆轮逆运动学分配为 LF、LB、RF、RB 四个轮速目标。x、y 方向分别设置运动学标定倍率，用于补偿轮径、安装角和底盘结构造成的方向比例差异；旋转项使用 $L=HALF_X_AXIS+HALF_Y_AXIS=19\text{ cm}$ 。

$$v_{LF}=v_y+v_x-L\omega, v_{LB}=v_y-v_x-L\omega, v_{RF}=v_y-v_x+L\omega, v_{RB}=v_y+v_x+L\omega \quad \text{公式 1}$$

逆运动学在 20 ms 慢环产生新的轮速和目标加速度。若直接把该阶跃交给 5 ms 速度环，会周期性激励电机和底盘，因此系统把相邻慢环目标差分为四拍，在 4 个快环周期内线性铺开。这样既保持 200 Hz 轮速闭环的响应速度，又减小切段、转向和刹车时的目标跳变。

四个车轮分别使用独立 PID 参数，不假设电机和机械阻力完全一致。控制输出由速度前馈、静摩擦补偿、加速/制动前馈和 PID 校正共同组成：kv 提供与目标轮速成比例的基础占空比，ks 用平滑符号函数克服低速静摩擦，ka 与 kb 分别补偿加速和制动阶段，PID 负责修正模型误差和负载变化。

$$u_i=k_v \cdot v_i^*+k_s \cdot \text{smooth_sign}(v_i^*)+k_a/k_b \cdot a_i^*+K_p \cdot e_i+I_i+D_i \quad \text{公式 2}$$

积分项只在目标加速度和速度误差较小时累积，大加减速阶段则逐步退避，防止起步阶段的历史积分在停车时释放为过冲；积分输出限制在固定范围内实现抗饱和。微分项不直接对速度误差差分，而是比较实测加速度与目标加速度，

经低通滤波后只阻尼计划外加速度，避免抵消 k_a 、 k_b 前馈。目标、实测轮速和目标加速度均接近零时关闭输出，减少静止状态下由摩擦补偿引起的爬行。

表 5.2 底盘连续控制链路

控制层	主要输入	算法	主要输出
路径层	网格路径、逻辑起点	共线压缩与航点管理	物理航点
轨迹层	航点、当前位置	线跟踪、前瞻与速度规划	全局 v_x 、 v_y
姿态层	目标 yaw、IMU 角速度	限速角速度规划与阻尼	ω
运动学层	车体 v_x 、 v_y 、 ω	麦克纳姆逆运动学	四轮目标速度
执行层	目标/实测轮速与加速度	前馈 + 独立 PID	四路 PWM

5.7 航向、位姿融合与安全控制

YawProfiled 将航向误差归一化到最短旋转方向，并根据误差大小规划角速度和角加速度。远离目标角时使用受最大角速度约束的快速旋转，接近目标后进入线性区；IMU 实测角速度作为阻尼项抑制冲过和回摆。车辆平移时使用较保守的航向增益，观测点到位后再解除 `hard_lock` 原地对准，角度进入容差且车辆完全停稳后才允许拍照。

位姿估计以编码器为高频短时基准。四轮实测速度通过正运动学得到车体 v_x 、 v_y 和角速度，结合 IMU yaw 旋转到全局坐标，并以 20 ms 周期积分位置。系统分别保存 x、y 方向的里程比例，视觉条件稳定时可依据一段直线运动中的视觉位移和编码器位移缓慢学习比例，以补偿长期轮径误差。

ART1 位姿具有通信和图像处理延时，不能直接覆盖当前编码器位置。PoseEstimator 保存里程计历史，根据配置的视觉延时回到图像采集时刻，再将采集后累计的编码器位移补偿到当前时刻。修正量按当前路径段分解为纵向和横向：横向误差在全段限步矫正，纵向误差在中远段缓慢修正，进入刹车与终点冻结窗口后停止注入视觉位置，避免延时旧帧破坏停车判定。偏差过大时执行硬重置，异常帧则直接拒绝。

上述视觉—里程计融合也是解决实车左右平移前后漂移的主要软件手段。编码器只能反映轮子转动，无法辨别底盘变形或载荷不对称造成的轮胎滑移；

视觉坐标能够观测真实位移，因此控制系统用视觉限制累计误差，同时保留低延时编码器负责末端收尾。该方法不能替代底盘机械校正，但能够在机械问题尚未完全排除时提高任务完成稳定性。

安全控制贯穿跟踪和状态机。四轮目标速度与实测速度均小于 1.5 cm/s 才判定车辆停稳；位置明显越出 240 cm×320 cm 场地时立即切换到 POINT_TRACKING 并锁定当前位置；规划失败、语义不足或视觉帧过期时不继续派发危险动作。推炸弹后还设置爆炸等待和视觉抑制窗口，FINISHED 与 ERROR_OCCURRED 状态均将底盘目标固定在当前位置。

第六章 开发、制作与调试

6.1 开发工具

目标工程使用 Keil MDK 与 armclang 构建，工程文件为 project/mdk/rt1064.uvprojx。桌面 g++ 仅用于算法语法检查，simulation 目录提供地图、仿真入口和回归脚本。最终版本必须通过 Keil 编译并检查 map 文件中的 ITCM、DTCM 和 OCRAM 占用。

6.2 安装与调试过程

- 单独核验电机方向、编码器正负和四轮速度闭环
- 完成直线、横移和原地旋转标定，再联调路径跟踪
- 用 Mock 模式验证地图、巡图、语义绑定与推箱状态转移
- 接入 ART1 后验证超时重发和稳定帧筛选，接入 ART2 后验证 ACK 与请求掩码
- 每次修改大型缓存后重新检查 Keil map 文件和静态内存

6.3 故障保护

表 6.1 故障场景与保护动作

故障场景	保护动作
地图帧超时	每 1 s 重发 ART1 地图请求
视觉位姿过期或不稳定	不应用修正并继续取样
炸弹爆炸窗口	等待并暂时抑制视觉修正
规划失败	记录 error_stage 并停车保持
位姿越界	切换 POINT_TRACKING 并锁定当前位置
语义不完整	继续巡图或重新感知

6.4 测试记录

表 6.2 实车测试记录模板

测试项目	测试条件	结果数据	结论
直线定位	100 cm, 空载	待填写	待填写
横移定位	80 cm, 空载	待填写	待填写

第二十一届全国大学生智能汽车竞赛技术报告

测试项目	测试条件	结果数据	结论
原地旋转	90°/180°	待填写	待填写
视觉位姿融合	连续稳定帧	待填写	待填写
炸弹清障	仿真及实车地图	待填写	待填写
完整赛程	Prod 模式	待填写	待填写

第七章 主要技术参数

表 7.1 汇总源码能够确认的技术参数。整车尺寸、质量、功耗、总电容和视觉检测指标必须在实车测试后填写，不能用估计值替代。

表 7.1 模型车主要技术参数

类别	参数	数值或状态
主控	MCU	NXP i.MX RT1064, Cortex-M7, 600 MHz
地图	尺寸	12×16 格, 20 cm/格, 240×320 cm
规划	最大箱子/炸弹	10 / 3
规划	MAX_PATH_LENGTH	规范目标 200; 当前源码 340, 提交前统一
底盘	轮半径	3.15 cm
底盘	半轴距	X=9.0 cm, Y=10.0 cm
编码器	线数/倍频/传动比	1024 / 4 / 70:30
通信	ART1/ART2	115200 bps
控制	轮速/底盘/IMU 周期	5 ms / 20 ms / 1 ms
结构	长、宽、高和总质量	待实测
电路	系统功耗和总电容	待实测
传感器	种类与数量	编码器 4、IMU1、 OpenART2; 按实车复核
执行器	额外伺服电机数量	待实车确认
视觉	检测精度和频率	待实测

第八章 结论

本模型汽车已经完成从机械装配、传感器安装、电气连接到控制软件集成的整体制作。机械部分采用四轮麦克纳姆轮底盘，电气部分以 NXP i.MX RT1064 为主控并连接四路电机、编码器、ICM42688、双 OpenART 和 TFT；软件部分实现了视觉通信、位姿估计、巡图规划、炸弹清障、语义绑定、推箱求解、路径跟踪和异常停车。整车已经形成“出库建图—巡图识别—语义匹配—推箱执行—返航”的完整任务链路，并可通过 Mock 和 Demo 模式在脱离部分硬件输入时检查状态机和算法流程。

由当前硬件配置和源码能够确认的主要技术指标包括：主控为 600 MHz Cortex-M7 内核的 RT1064；比赛地图按 12×16 网格表示，单格尺寸为 20 cm；系统最多处理 10 个箱子和 3 个炸弹；ART1 与 ART2 均采用 115200 bps UART 通信；IMU 更新周期为 1 ms，四轮速度闭环周期为 5 ms，视觉位姿修正周期为 10 ms，底盘控制和里程计更新周期为 20 ms。整车尺寸、质量、系统功耗、总电容、视觉识别准确率和完整赛程耗时仍需以最终实车测量结果为准。

实际调车过程中遇到的最突出问题是编码器点位不准确，车辆进行左右平移时会同时产生前后方向偏移。调试中先后检查并更换了相关编码器与硬件，也调整了编码器方向、轮速闭环、运动学参数和控制代码，但该现象仍未得到根本解决。结合不同方向和不同速度下的重复测试，问题可能并非单一编码器或软件计算错误，而是底盘结构发生变形，使四个麦克纳姆轮的接地状态、法向载荷或几何位置不再对称，进而产生方向不一致的打滑。该问题说明仅依赖编码器里程计难以区分真实位移与轮胎滑移，继续单纯调整 PID 参数无法从根本上消除系统性偏移。

现阶段采用 ART1 视觉坐标周期性矫正编码器里程计，以限制左右平移误差的累计。后续应检查车架刚度、轮轴平行度、轮组安装角度和四轮载荷，恢复底盘机械对称性，再分别标定各方向的有效轮径和滑移系数。还可以增加负压装置，提高车辆附着并减小打滑，同时记录不同速度、方向和电量条件下的视觉—里程计误差，建立方向补偿模型。我们的判断是，应先消除机械结构造成的非对称误差，再用视觉闭环和软件补偿处理剩余扰动，而不能依靠不断增加控制增益掩盖机械问题。

参考文献

- [1] 全国大学生智能汽车竞赛组委会. 第二十一届全国大学生智能汽车竞赛技术报告要求细则[EB/OL]. 2026.
- [2] NXP Semiconductors. i.MX RT1060 Reference Manual[Z].
- [3] 全国大学生智能汽车竞赛组委会. 全国大学生智能汽车竞赛比赛规则与赛道说明[Z].
- [4] Intelligent-Vision 项目源代码: project/Algorithm、project/Subsystem、project/Device、project/Core[Z].

附录 A 核心程序

以下列出调度器和视觉帧发送核心程序。完整工程源代码随参赛作品一并提交。

```
void Core::Scheduler::run() {
    uint32_t current_time = get_sys_tick_ms();
    for (uint8_t i = 0; i < TASK_COUNT; ++i) {
        if (current_time - task_table[i].last_run_ms >=
            task_table[i].interval_ms) {
            task_table[i].last_run_ms = current_time;
            task_table[i].task_func();
        }
    }
}

void UartComm::send_packet(uint8_t type,
                           const uint8_t* payload, uint8_t len) {
    uint8_t header[4] = {0xAA, 0x55, type, len};
    uint8_t checksum = type + len;
    for (int i = 0; i < len; ++i) checksum += payload[i];
    uart_write_buffer(uart_idx_, header, 4);
    if (len > 0) uart_write_buffer(uart_idx_, payload, len);
    uart_write_byte(uart_idx_, checksum);
}
```

附录 B 电路板原理图

此处必须插入最终版电路板详细设计原理图，包括主控连接、电源、电机驱动、编码器、IMU、双 OpenART、TFT 和保护电路。仓库中未提供可核验的原理图文件，因此本报告不虚构电路连接。提交前请用最终原理图替换本说明。

附录 C 提交前检查表

- 填写学校、队伍名称、参赛队员和带队教师
- 签署授权说明
- 补充机械安装照片和相机安装尺寸
- 插入最终电路板详细原理图
- 填写整车尺寸、质量、功耗、总电容、伺服电机数量和视觉指标
- 统一 MAX_PATH_LENGTH 并完成算法回归
- 完成 Keil 编译，核对 ITCM、DTCM 和 OCRAM 占用
- 在 Word 中更新目录、页码及所有域